

Identificação do Aluno**Nome:** Vinicius Reis Gonçalves**Nº USP:** 15491921

Formato de entrega: Responda as questões nesse arquivo .docx, salve e **envie em formato .pdf**. Caso não utilize o Microsoft Word, crie seu modelo (seguindo esse formato) no editor de texto de sua escolha. Nomeie o arquivo da seguinte forma: **SEL0615_EX<número>_<SeuNome>.pdf**

EXERCÍCIO 2

1) Dado o script de DFT da Aula 3, implemente-o em Matlab ou Python. Na sequência,

complemente a implementação para:

a) Ler o sinal do arquivo Sinal.txt e sua estampa de tempo do arquivo TimeStamp.txt. Cada elemento do sinal e da estampa de tempo está separado por ponto e vírgula.

```
# Abre os arquivos de amplitude do sinal e do tempo discreto no modo leitura
sinal_file = open("Sinal.txt","r")
time_file = open("TimeStamp.txt","r")

# Lê tudo e separa pelo delimitador ';' -> Transforma em uma lista de strings
val_list = sinal_file.read().split(';')
time_list = time_file.read().split(';')

# Converte de str para float cada elemento da lista
val_list = [float(x) for x in val_list]
time_list = [float(x) for x in time_list]
```

b) Determinar a frequência de amostragem desse sinal.

```
# Número total de amostras
smp1_numb = val_list.__len__()

smp1_freq = 0.0
total_time = 0

# Percorre até o penúltimo elemento do vetor de timestamp
for i in range(0,smp1_numb - 1):

    # Soma todos os períodos entre amostragens
    total_time += time_list[i+1] - time_list[i]

# Divide o período total de coleta pelo n° de amostras (Tamanho médio de cada período, descontando a primeira amostragem)
total_time = total_time/float(smp1_numb - 1)

# Frequência de amostragem == 1/período médio
smp1_freq = 1/(total_time)

print(f"Frequência de amostragem: {smp1_freq:0.2f} Hz")
```

Frequência de amostragem == 3840 Hz

c) Determinar as frequências dos harmônicos presentes nesse sinal.

```

# Aplica a dft
X = fft(val_list)

# Cria um vetor com tamanho do número de amostras cujo espaçamento entre passos é a frequência de amostragem/número de amostras
freqs = np.arange(smpl_numb) * smpl_freq / smpl_numb

# Pega apenas o semiciclo positivo do sinal
half = smpl_numb//2
pstv_half = X[:half]

freqs_half = freqs[:half]

# Pega apenas as frequências positivas
amplitudes = (2/smpl_numb) * np.abs(pstv_half)

# Define o threshold como 1% da amplitude mais alta
threshold = float(0.01 * np.max(amplitudes))

# Pega os índices onde as amplitudes são maiores que os thresholds (frequências harmônicas)
harmonic_idx = np.where(amplitudes >= threshold)[0]

# Exclui o nível DC da análise e guarda apenas as frequências harmônicas
harmonic_freq = freqs_half[harmonic_idx]

```

Frequências harmônicas:

- 1° - 60 Hz;
- 3° - 180 Hz;
- 5° - 300 Hz;

d) Determinar as amplitudes desses harmônicos e da componente fundamental.

```

# Respectivas amplitudes das frequências harmônicas
harm_amps = amplitudes[harmonic_idx]

```

- 1° - 60 Hz - 129.90
- 3° - 180 Hz - 11.4
- 5° - 300 Hz - 2.10

Você deve enviar:

- Script implementado para responder (a)
- Arquivo pdf com as respostas de (b) a (d)